

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО»
Институт компьютерных наук и кибербезопасности

Отчет о прохождении производственной (технологической (проектно-технологической)) практики

Соловьева Дарья Александровна

(Ф.И.О. обучающегося)

3 курс, 5130203/30101

(номер курса обучения и учебной группы)

02.03.03 Математическое обеспечение и администрирование информационных систем

(направление подготовки (код и наименование))

Место прохождения практики: АО НПП «Интеграл-В»

(указывается наименование профильной организации или наименование структурного подразделения)

г. Санкт-Петербург, ул. Академика Павлова 14д

ФГАОУ ВО «СПбПУ», фактический адрес)

Сроки практики: с 15.06.2026 по 11.07.2026

Руководитель практической подготовки от ФГАОУ ВО «СПбПУ»: Пак Вадим Геннадьевич, к.ф.-м.н., доцент ВШТИИ

(Ф.И.О., уч. степень, должность)

Консультант практической подготовки от ФГАОУ ВО «СПбПУ»: нет

(Ф.И.О., уч. степень, должность)

Руководитель практической подготовки от профильной организации: Захарченко Владимир Сергеевич, главный специалист машинного обучения нейронных сетей АО НПП «Интеграл-В»

Оценка: отлично

Руководитель практической подготовки
от ФГАОУ ВО «СПбПУ»:

/Пак В.Г./

Руководитель практической подготовки
от профильной организации:

/Захарченко В.С./

Обучающийся:

/Соловьева Д.А./

Дата: 11.07.26

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. АНАЛИЗ МЕТОДОВ ИСКУССТВЕННОГО ИНТЕЛЕКТА РАЗДЕЛЕНИЯ ВИДЕОПОТОКА НА СЛОИ	4
1.1. SAM2 Video.....	4
1.2. TAP - Tracking Any Point	4
1.3. XMem.....	5
1.4. Сегментация с помощью Bbox	6
1.4.1. OTrack.....	6
1.4.2. NanoTrack	6
1.4.3. ATOM	7
1.5. Omnimate-sp	7
1.6. Сравнительный анализ выбранных методов.....	7
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ РАЗДЕЛЕНИЯ ВИДЕО НА СЛОИ	9
2.1. Архитектура веб-приложения	9
2.2. Реализация процессоров разделения на слои.....	10
2.2.1. Процессор SAM2 Video.....	10
2.2.2. Процессор TAP.....	11
2.2.3. Процессор OTrack	11
2.2.4. Процессор NanoTrack.....	12
2.2.5. Процессор XMem	12
2.3. Разработка пользовательского интерфейса на Gradio	13
ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ ЭФФЕКТИВНОСТИ МЕТОДОВ	16
3.1. Получение точек объекта на первом кадре	16
3.2. Метрики оценки	16

3.2.1. Характеристики исходного видео	17
3.2.2. Программные метрики	17
3.2.3. Качественные метрики результата обработки.....	18
3.3. Результаты бенчмаркинга методов	19
3.4. Сравнение результатов обработки методов	22
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСТОЧНИКОВ.....	27
Приложения	29

ВВЕДЕНИЕ

Современные технологии компьютерного зрения и машинного обучения открывают новые возможности для обработки видеоданных. Одной из активно развивающихся областей является разделение видео на слои с помощью нейросетей. Необходимо выделить отдельные объекты видеопотока (например, людей, животных или транспорта) отделить их от фона и друг от друга. Такие технологии востребованы в системах видеонаблюдения, при создании спецэффектов в кинопроизводстве, в дополненной и виртуальной реальности, а также в робототехнике и системах автономного вождения.

В рамках производственной практики передо мной была поставлена задача разработки веб-приложения для разделения видеопотока на слои с использованием современных нейросетевых моделей.

Объектом исследования являются методы искусственного интеллекта для сегментации объектов в видеопотоке. Предмет исследования: программная реализация веб-приложения для обработки видеоданных.

Цель практики: сравнение различных нейросетевых методов разделения видео на слои.

Для достижения поставленной цели были определены следующие задачи:

- Провести аналитический обзор современных методов сегментации видео и обосновать выбор технологий.
- Спроектировать архитектуру веб-приложения и реализовать процессоры для каждого из выбранных методов разделения на слои.
- Разработать пользовательский интерфейс на базе и реализовать процессоры для каждого из выбранных методов разделения на слои.
- Провести экспериментальное исследование разработанных методов, оценив их производительность и качество сегментации на различных видеоматериалах и разрешениях.
- Сравнить методы между собой и сформулировать рекомендации по их применению в зависимости от сценария использования.

Таким образом, в рамках данной практики планируется не только реализация веб-приложения, но и анализ производительности и эффективности нейросетевых методов разделения на слои.

ГЛАВА 1. АНАЛИЗ МЕТОДОВ ИСКУССТВЕННОГО ИНТЕЛЕКТА

РАЗДЕЛЕНИЯ ВИДЕОПОТОКА НА СЛОИ

1.1. SAM2 Video

SAM2 (Segment Anything Model 2) – это базовая модель для решения задачи быстрой визуальной сегментации изображений и видео, разработанный Meta [1]. SAM2 Video – это метод на основе модели SAM2, адаптированной для видео, универсальная модель для сегментации объектов в видео, которая умеет работать как с промптами (точки, боксы), так и в полностью автоматическом режиме (рис. 1.1.1).

Модель отлично определяет объекты благодаря механизму памяти, состоящий из 3 параметров:

- Memory Encoder кодирует информацию из текущего кадра и предыдущих предсказаний.
- Memory Bank хранит «воспоминания» о недавних кадрах и объектах, включая информацию о движении.
- Memory Attention Module позволяет текущему кадру «вспоминать» контекст прошлых: модель учитывает, где объект был раньше, чтобы точнее предсказать его положение сейчас.

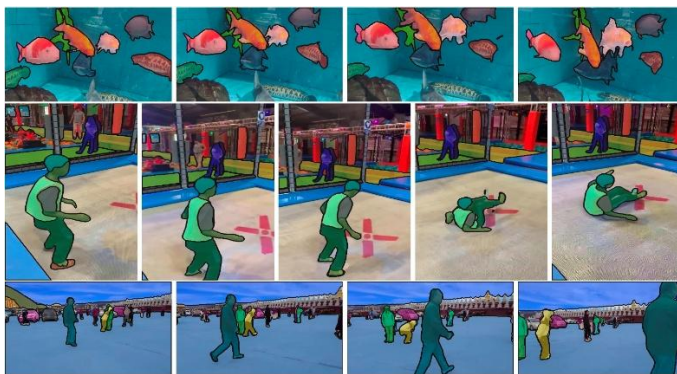


Рис. 1.1.1. Применение SAM2 к кадрам видео

Метод показывает выдающуюся точность, но требует огромных вычислительных ресурсов (видеокарты с 24 ГБ+).

1.2. TAP - Tracking Any Point

Метод Tracking Any Point решает задачу через призму трекинга физических точек на поверхности объектов, а не классической маски [2]. Вместо выделения «области» он учится предсказывать, куда переместится конкретный пиксель через N кадров, вычисляя траектории (рис. 1.2.1).

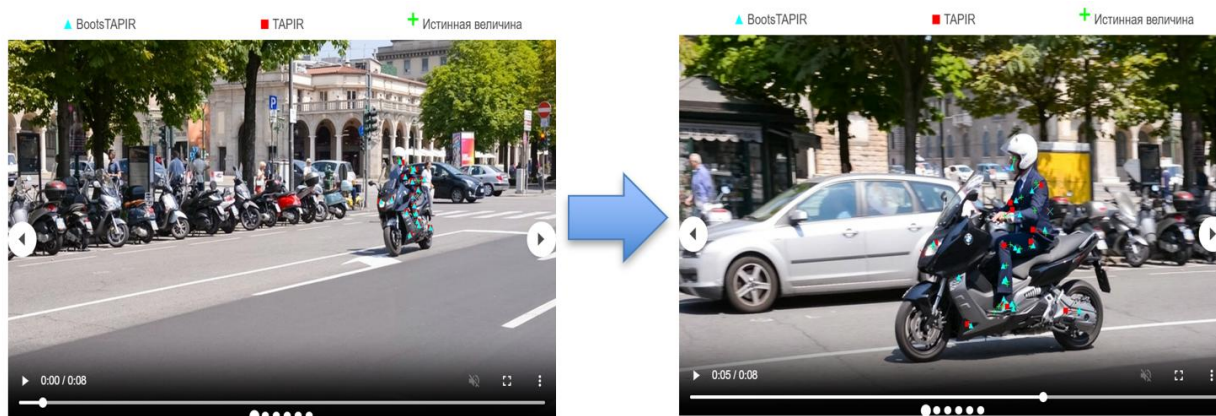


Рис. 1.2.1. Пример трекинга точек на видео

Такой подход дает высокую точность динамики объекта, но не выдает готовых слоев-масок, требует постобработки (кластеризации точек) и может «терять» точки при резких сменах освещения или текстуры объекта.

1.3. XMem

Модель XMem рассматривает сегментацию видеообъектов (VOS) как проблему управления памятью для построения маски видео (рис. 1.3.1). В отличие от предшественников, использовавших единственный тип хранилища (веса, последний кадр, скрытые состояния или пространственные признаки), этот подход учитывает, что краткосрочные решения нестабильны, а долгосрочные – чрезмерно затратны по ресурсам и часто теряют разрешение из-за сжатия признаков (например, AFB-URR) [3].

Метод базируется на трёхкомпонентной модели памяти Аткинсона-Шиффрина (сенсорная, рабочая и долговременная память). Разные временные масштабы этих хранилищ интегрированы в механизм считывания, что позволяет эффективно обрабатывать как короткие сцены, так и видео длиной свыше 10 000 кадров без потери качества и без экспоненциального роста вычислений.



Рис. 1.3.1. Пример построения маски методом XMem

Для каждой конкретной программы можно настроить вручную десятки параметров отвечающие за память, хранение кадров, частоту и качество сохраняемых данных, что позволяет подобрать оптимальный баланс между скоростью работы и качеством результата.

1.4. Сегментация с помощью Vbox

Bounding Box (ограничивающий прямоугольник) – это основной инструмент в задачах компьютерного зрения, представляющая собой прямоугольную рамку, заданную координатами (x, y, ширина, высота), которая выделяет интересующий объект на изображении. На рисунке 1.4.1 белые контуры это маски объекта, соответствующие его формам, а красные прямоугольники – это соответствующие Vbox.



Рис. 1.4.1. Сравнения Vbox и маски на различных формах

Строится и отслеживается Vbox проще и быстрее чем полноценная точная маска, но может терять объект из-за лишнего шума фона.

1.4.1. OSTrack

OSTrack (One-Stream Tracker) – это новейший подход на базе трансформеров [4]. Объединяет извлечение признаков (поиск объекта) и взаимное сопоставление (сравнение с шаблоном) в единую сеть (One-Stream). Использует визуальные трансформеры, чтобы глобально моделировать зависимости между объектом и областью поиска. Работает быстро и очень точно, так как все вычисления идут «в одном потоке».

1.4.2. NanoTrack

NanoTrack снован на сверточных сетях (CNN) и построен по принципу Siamese Network (две ветви: шаблон и поисковая область) [5]. Главной особенностью является миниатюрная архитектура (MobileNet-like), которая позволяет работать на мобильных устройствах и CPU со скоростью сотни FPS. Точность средняя, но упор сделан на скорость и малый вес модели.

1.4.3. ATOM

ATOM (Accurate Tracking by Overlap Maximization) является классическим гибридным методом, состоящим из двух модулей:

- Оценка грубого положения (быстрая Siamese сеть).
- Уточнение точного положения (специальная модульная сеть, которая обучается предсказывать пересечение с эталоном IoU).
- ATOM фокусируется на качестве наложения (IoU), что делает его очень устойчивым к изменению масштаба и позы объекта. Точность высокая, но работает медленнее современных трансформеров.

Официальная реализация ATOM находится в общем фреймворке PyTracking [6].

1.5. Omnimate-sp

Название метода Omnimate происходит от сочетания слов omni (всеобъемлющий) и matte (матовое покрытие, в компьютерной графике маска для выделения объектов).

В отличие от классической сегментации, которая просто выделяет контур объекта, Omnimate решает более сложную и интересную задачу: разложение видео на слои, где каждый слой содержит не только сам объект, но и все связанные с ним динамические эффекты: тени, отражения, дым, брызги и т.д. Это принципиально важно, так как при обычной сегментации эффекты либо игнорируются, либо ошибочно приписываются фону.

Метод требует на вход не только исходное видео и инициализирующую маску или координаты, но и грубые маски сегментации для одного или нескольких объектов. Для каждого объекта создается слой в формате RGBA.

1.6. Сравнительный анализ выбранных методов

Сравнение методов представлено в таблице 1.5.1.

Таблица 1.6.1. Сравнительный анализ методов сегментации и трекинга видеопотока

Метод	Скорость обработки	Точность	Требования	Сложность реализации
SAM2 Video	Низкая	Очень высокая	Очень высокие	Высокая
TAP	Средняя	Высокая	Высокие	Средняя
XMem	Средняя / Высокая	Высокая	Средние (настраиваемые)	Средняя
OSTrack	Высокая	Высокая	Высокие	Высокая
NanoTrack	Очень высокая	Средняя	Очень низкие	Низкая
ATOM	Низкая	Высокая	Средние	Средняя
Omnimatte	Низкая	Очень высокая	Очень высокие	Высокая

На основе сравнительного анализа были выбраны следующие модели:

- SAM2 Video и XMem как модели для задач высокоточной интерактивной сегментации.
- OSTrack, NanoTrack как модели для обработки систем реального времени на основе Vbox
- Метод TAP как специализированное решение для задач анализа траекторий движения.

Omnimatte-sp уникален для задач разложения сцены на слои с сохранением эффектов окружения, однако он требует на вход уже обработанный поток, поэтому не будем включать его в перечень реализуемых моделей для сравнения.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ РАЗДЕЛЕНИЯ ВИДЕО НА СЛОИ

2.1. Архитектура веб-приложения

Структура проекта представлена на рисунке 2.1.1.

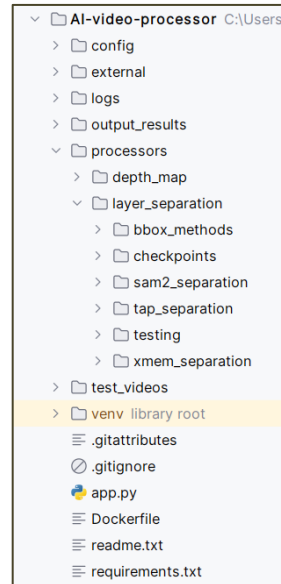


Рис. 2.1.1. Структура проекта

Проект запускается с помощью файла *app.py*, в котором инициализируется Gradio-интерфейс, связываются пользовательские действия (загрузка видео, выбор точек на первом кадре, запуск обработки) с соответствующими процессорами и управляет отображением результатов.

В папке *processors* находятся все файлы отвечающие за обработку видео: папки *depth_map* и *layer_separation*. Последняя содержит реализации пяти различных алгоритмов:

- *sam2_separation/* (файлы: *layer_separation.py*, *sam2_segmenter.py*).
- *tap_separation/* (файлы: *tap_processor.py*, *cotracker_wrapper.py*).
- *bbox_methods/* (*ostrack_processor.py* и *nanotrack_processor.py*).
- *xmem_separation/* (файл: *xmem_processor.py*)
- *checkpoints/* – хранилище весов предобученных моделей для всех вышеперечисленных методов (файлы *.pt*, *.pth*, *.onnx*, *.yaml*).

Также директория проекта содержит:

- `test_videos/` для загрузки и хранения исходных видео
- `output_results/` для сохранения результатов обработки
- `logs/` хранилище лог-файлов приложения.

В папке `config` файл `config.yaml` хранит все настройки приложения (пути к директориям, параметры моделей, устройство вычислений, размеры чанков и т.д.), файл `config_settings.py` загружает YAML-конфиг, создаёт объекты путей и настраивает глобальное логирование.

2.2. Реализация процессоров разделения на слои

Многие из реализуемых методов могут строить маски без исходных точек, однако так как стоят требования отслеживать конкретный выбранный объект, то на вход всем процессам передаются координаты точек объекта. Результатом являются два видеофайла: слой объекта и фоновый слой.

2.2.1. Процессор SAM2 Video

Процессор реализован в классе `LayerSeparationProcessor` с использованием нативного видеотрекера модели SAM2 (*`build_sam2_video_predictor`*).

Видео разбивается на фрагменты фиксированной длины (`SAM2_CHUNK_SIZE`, по умолчанию 150 кадров). Для каждого чанка кадры сохраняются во временную директорию в формате JPG, так как SAM2 Video Predictor требует чтения данных из файловой системы и вызывается метод `process_video_tracking`, который инициализирует состояние (`inference_state`), добавляет стартовые точки на нулевом кадре чанка и запускает пропагацию маски через `propagate_in_video`.

После обработки текущего чанка процессор вычисляет центроид (среднее арифметическое координат) полученной маски объекта на последнем кадре чанка. Этот центроид передаётся в качестве начальной точки для

следующего чанка. Благодаря этому достигается плавный переход между чанками без потери объекта, даже при его движении или изменении формы.

Если в процессе обработки обнаруживается новый объектный слой, для него динамически создаётся новый VideoWriter и добавляется в список выходных файлов.

2.2.2. Процессор TAP

Этот процессор реализует трекинг на основе точек (point tracking) с помощью модели CoTracker.

Для первого кадра вызывается `sam2_segmenter.get_image_mask`, использующий `SAM2ImagePredictor` (в отличие от видеопредиктора). Это позволяет получить точную маску объекта. Из полученной маски извлекаются все пиксели, принадлежащие объекту. Чтобы избежать вычислительной нагрузки на CoTracker, координаты точек прореживаются с фиксированным шагом (`grid_step = 12`).

Для каждого чанка (размером 60 кадров) строится тензор видео и тензор запросов (координаты точек на нулевом кадре). Вызов `cotracker.track_chunk` возвращает траектории всех точек (`pred_tracks`) и булеву маску их видимости (`pred_vis`).

На каждом кадре из видимых (`vis > 0.5`) и успешно отслеженных точек строится выпуклая оболочка с помощью `cv2.convexHull`, которая затем заливается (`cv2.fillConvexPoly`) для получения бинарной маски объекта. Такой подход позволяет адаптивно изменять форму маски, но она всегда будет выпуклой.

2.2.3. Процессор OTrack

Этот процессор использует встроенный трекер OpenCV для отслеживания Bounding Box объекта.

На первом кадре строится маска через SAM2. Затем вычисляются минимальные и максимальные координаты x и y ненулевых пикселей маски, формируя прямоугольник (x_min , y_min , $width$, $height$).

Инициализируется трекер `cv2.TrackerViT_create()`. Vision Transformer (ViT) в составе этого трекера обеспечивает хорошую устойчивость к изменениям внешнего вида объекта. Поскольку поддержка TrackerViT зависит от сборки OpenCV, в реализации предусмотрен перехват исключения `AttributeError`. В случае недоступности ViT автоматически подключается более простой, но стабильный трекер `cv2.TrackerMIL_create()` без остановки выполнения программы.

2.2.4. Процессор NanoTrack

Инициализирующий Bbox строится аналогично процессору OTrack. Для создания трекера загружаются параметры `cv2.TrackerNano_Params()`, в которые явно передаются пути к файлам архитектуры: `backbone` (свёрточная основа) и `neckhead` (голова для классификации/регрессии). Оба файла имеют формат ONNX, что позволяет выполнять инференс на CPU с высокой скоростью. Как и в случае с OTrack, при ошибке создания NanoTrack (например, отсутствие модели на диске или ошибка совместимости) процессор переключается на TrackerMIL, логируя предупреждение.

2.2.5. Процессор XMem

На первом кадре маска строится SAM2 и передаётся в метод `processor.step(frame_tensor, mask_tensor)` для инициализации состояния памяти.

Для ускорения работы все кадры уменьшаются так, чтобы короткая сторона была равна `target_size = 360` пикселям. При этом ширина и высота дополнительно приводятся к кратности 16 (требование архитектуры XMem). Важно отметить, что итоговые видео сохраняются в оригинальном

разрешении, маска увеличивается обратно с помощью интерполяции cv2.INTER_NEAREST (для сохранения бинарности).

На рисунке 2.2.5.1 можно увидеть параметры моей реализации для баланса между точностью сегментации, скоростью обработки и потреблением памяти GPU.

```
self.config = {  
    'key_dim': 64, # Размерность ключей для поиска в памяти  
    'value_dim': 512, # Размерность значений (признаков объекта)  
    'hidden_dim': 64, # Размерность скрытых слоёв сети памяти  
    'single_object': False, # Режим множественных объектов (не один)  
    'top_k': 20, # Извлекать топ-20 похожих элементов из памяти  
    'mem_every': 20, # Добавлять кадр в память каждые 20 кадров  
    'deep_update_every': -1, # Глубокое обновление памяти отключено (-1)  
    'enable_long_term': True, # Включить долгосрочную память  
    'enable_long_term_count_usage': True, # Учитывать частоту использования элементов  
    'num_prototypes': 24, # Количество прототипов в долгосрочной памяти  
    'min_mid_term_frames': 10, # Минимум кадров в среднесрочной памяти  
    'max_mid_term_frames': 15, # Максимум кадров в среднесрочной памяти  
    'max_long_term_elements': 1500, # Максимум элементов в долгосрочной памяти  
}
```

Рис. 2.2.5.1. Параметры настройки модели XMem

Процессор использует смешанную точность (torch.amp.autocast) при работе на CUDA. Ядро InferenceCore автоматически управляет рабочей памятью (краткосрочной) и долгосрочной памятью. В коде реализована принудительная очистка кэша CUDA и сборка мусора (gc.collect()) каждые 100 кадров, а также периодическое логирование размера памяти (work_mem, long_mem) для мониторинга загрузки GPU.

2.3. Разработка пользовательского интерфейса на Gradio

Пользовательский интерфейс реализован с использованием библиотеки Gradio. Ключевым архитектурным решением является использование единственного глобального экземпляра класса SAM2Segmenter для всех процессоров для экономии памяти и времени работы программы.

Интерфейс разделён на две колонки: левая колонка содержит компоненты загрузки видео (gr.Video), правая колонка отображает

результаты: выпадающий список полученных видеофайлов и плеер для просмотра выбранного слоя (рис. 2.3.1).

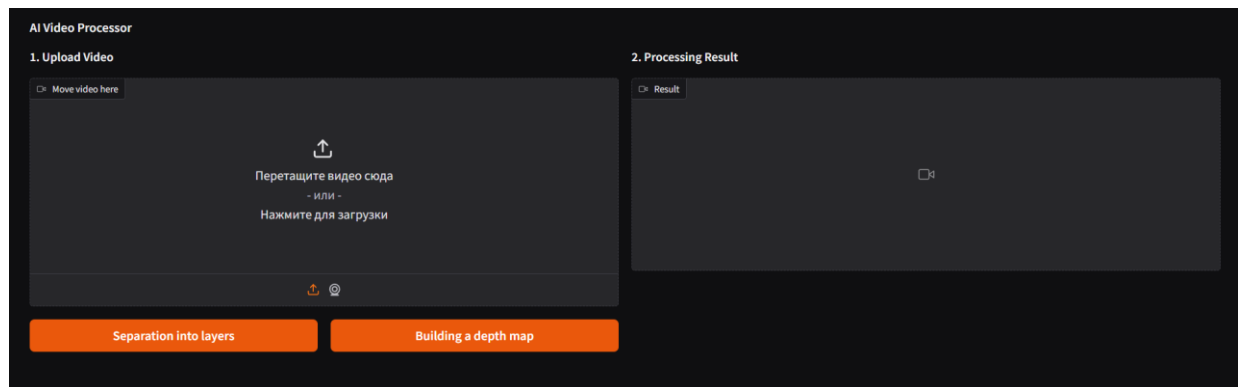


Рис. 2.3.1. Демонстрация интерфейса загрузки видео

После загрузки видео открывается радиокнопка (`tracking_method`) с пятью вариантами (SAM2 Video, TAP, OTrack, NanoTrack, XMem) и редактор изображений (`gr.ImageEditor`) с первым кадром видео. В редакторе пользователь отмечает красным цветом координаты объекта, которые будут использоваться для построения инициализирующей маски видео (рис. 2.3.2). Далее пользователь нажимает `Start Object Tracking` (`run_track_btn`) и запускается процесс обработки видео выбранным методом.

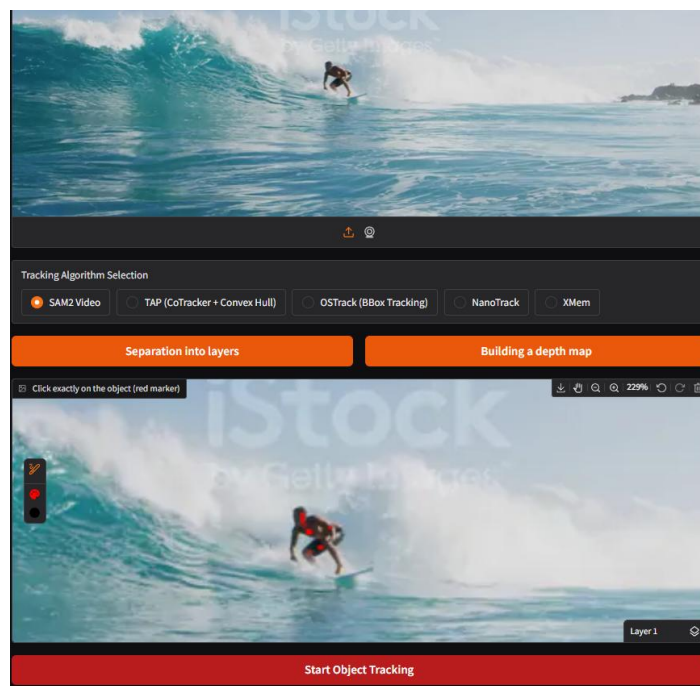


Рис. 2.3.2. Демонстрация интерфейса редактирования первого кадра

По завершении обработки видео выводятся 2 слоя видео: вырезанный объект (на черном фоне) и фон (рис. 2.3.3). Переключать слои можно с помощью выпадающего списка. Оба видео можно проиграть и скачать.

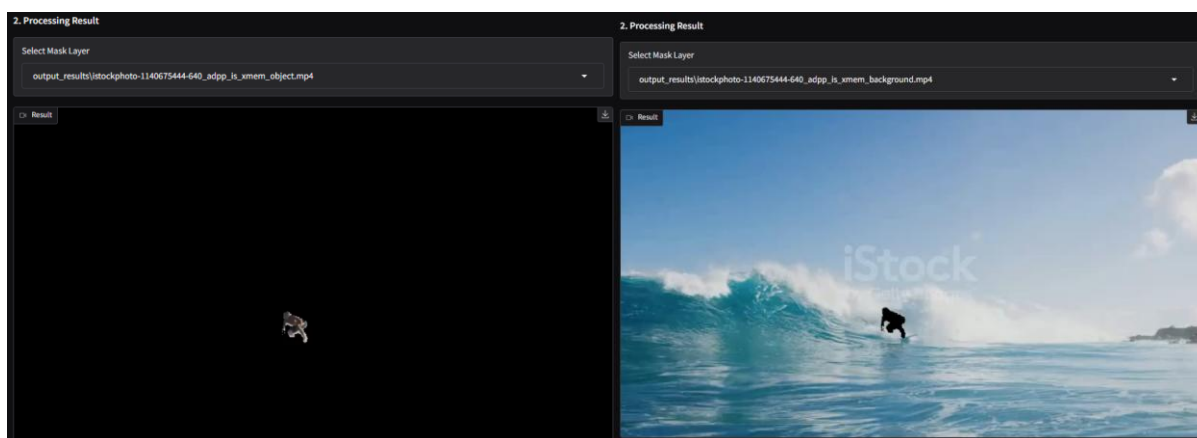


Рис. 2.3.3. Демонстрация вывода выделенных слоев видео

Приложение запускается через стандартный блок `if __name__ == "__main__"`. Параметры командной строки (`--ip` и `--port`) позволяют гибко настраивать сетевой адрес и порт сервера. По умолчанию сервер слушает все интерфейсы (0.0.0.0) на порту 7860. Все действия логируются с детализацией ошибок и предупреждений в файл `logs/app.log` и консоль.

ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ ЭФФЕКТИВНОСТИ МЕТОДОВ

3.1. Получение точек объекта на первом кадре

Для сравнения эффективности и производительности метрик созданы скрипты `benchmark.py` для запуска обработки нескольких видео каждым методом и `annotate_points.py` для получения инициализирующих точек.

Скрипт `annotate_points.py` запускает сайт Gradio, который отображает видеофайлы, размещённые в локальной папке `videos`. На первом кадре каждого видео пользователь ставит красные точки на объекте, который требуется отслеживать (рис. 3.1.1). Координаты точек сохраняются в JSON-файл с именем `{имя_видео}_points.json` рядом с исходным видео.

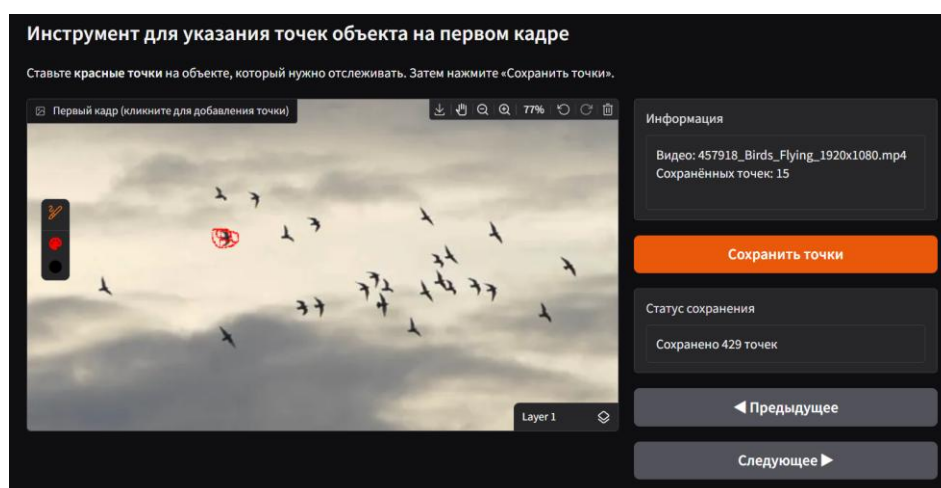


Рис. 3.1.1. Страница сохранения исходных точек для всех видео

Далее для запуска каждого метода координаты из соответствующих файлов `json` используются для построения первой маски или `bbox`.

3.2. Метрики оценки

При программном сравнении моделей сегментации фиксируются: характеристики исходного видео, программные характеристики во время работы модели и качественные характеристики результата обработки (без использования эталонной маски).

3.2.1. Характеристики исходного видео

Для анализа зависимости производительности и эффективности методов от исходных данных фиксируются следующая информация:

- Ширина и высота кадра (video_width, video_height)
- Частота кадров (video_fps)
- Общее количество кадров (video_frame_count)
- Длительность в секундах ($\text{video_duration_sec} = \text{frame_count} / \text{fps}$)
- Размер файла (video_size_mb)
- Количество точек из JSON-файла (num_points_original)
- Количество точек после сэмплирования, если исходных больше лимита метода (num_points_used).

Все эти метрики извлекаются на этапе подготовки (функция get_video_info) и записываются в отчёт для каждого запуска метода, что позволяет оценивать результаты в совокупности с характеристиками входного видео.

3.2.2. Программные метрики

Для анализа производительности и ресурсоёмкости фиксируются следующие метрики:

- Время выполнения в секундах (time_sec)
- Использование оперативной памяти (RAM)
 - ram_before_mb – объём RAM, занимаемый процессом до начала обработки;
 - ram_after_mb – объём RAM после завершения;
 - ram_delta_mb = ram_after - ram_before – изменение памяти.
- Использование видеопамати (VRAM)
 - vram_before_mb – выделенная память CUDA до запуска;
 - vram_after_mb – после завершения;
 - vram_delta_mb – изменение;

- `vram_peak_mb` – пиковое значение выделенной памяти за время работы.
- Количество созданных выходных файлов ненулевого размера (`output_files_created`)

3.2.3. Качественные метрики результата обработки

Данные метрики вычисляются путём сравнения соседних кадров и анализа геометрии бинарных масок из обработанного видео. Они не требуют эталонов и позволяют оценить временную стабильность и форму полученных масок.

1. Средняя компактность (`mean_compactness`)

Для каждого кадра, где площадь маски $area > 0$, вычисляется компактность по формуле: $C = 4\pi \cdot area / (perimeter^2)$. Периметр находится через контуры (`cv2.findContours`). Компактность принимает значения от 0 до 1: чем ближе к 1, тем форма ближе к окружности (более компактна). По всем кадрам усредняется.

2. Максимальное падение площади (`max_area_drop`)

Характеризует резкие изменения размера объекта между соседними кадрами. Вычисляется как максимальное значение относительного уменьшения: $drop_i = \max(0, -(area_{i+1} - area_i) / area_i)$. Если знаменатель равен нулю, он заменяется на 1 во избежание деления на ноль. Метрика показывает наибольшее относительное сокращение площади за один шаг по времени.

3. Временной мерцание (`temporal_flicker`)

Оценивает, насколько сильно меняется периметр маски от кадра к кадру. Рассчитывается как среднее относительное изменение периметра:

$$F = \frac{1}{N-1} \sum_{i=1}^{N-1} \frac{|P_{i+1} - P_i|}{P_i},$$

где N – общее число кадров в видео, P_i – периметр сегментированной области на i -м кадре.

Знаменатель заменяется на 1 при нуле. Меньшие значения означают более плавное изменение границ во времени.

3.3. Результаты бенчмаркинга методов

Для сравнения методов были выбраны 6 случайных видео. Одно из них (Birds_02__4K_res.mp4) имело расширение 4к. С помощью скрипта scale_video.py были получены 4 его копии имеющие соответственно расширения: 1280x720, 854x480, 640x360, 426x240. Итого стало 10 видео для бенчмаркинга. На таблице 3.3.1.1 представлены характеристики всех видео.

Таблица 3.3.1. Характеристики исходных видео

Видео	Разрешение	FPS	Кадров	Длит., с	Размер, МБ	Исходных точек
457918_Birds_Flying_1920x1080.mp4	1920×1080	25	280	11,2	14	15
Birds_02__4K_res.mp4	4096×2160	23,98	258	10,76	13,54	105
Birds_02__4K_res_1280x720.mp4	1280×674	23,98	258	10,76	0,44	82
Birds_02__4K_res_426x240.mp4	426×224	23,98	258	10,76	0,06	7
Birds_02__4K_res_640x360.mp4	640×336	23,98	258	10,76	0,1	18
Birds_02__4K_res_854x480.mp4	854×450	23,98	258	10,76	0,2	17
field.MOV	352×416	30	348	11,6	1,05	143
istockphoto-1140675444-640_adpp_is.mp4	768×432	23,98	245	10,22	1,66	68
istockphoto-1304919722-640_adpp_is.mp4	768×432	23,98	340	14,18	2,1	496
ivan.MOV	720×1280	60,17	254	4,22	1,53	970

Анализ производительности и скорости обработки представлен в приложениях 1 и 2, по ним был построен график 3.3.1. Метод NanoTrack является самым быстрым, обеспечивает в среднем 84,81 кадра в секунду. Модели OSTRack и XMem показывают схожую производительность (около 21 кадра в секунду). Однако XMem генерирует полноценную маску, а OSTRack всего лишь bbox. Методы TAP и SAM2 работают медленнее всего (от 1 до 3 кадров в секунду).

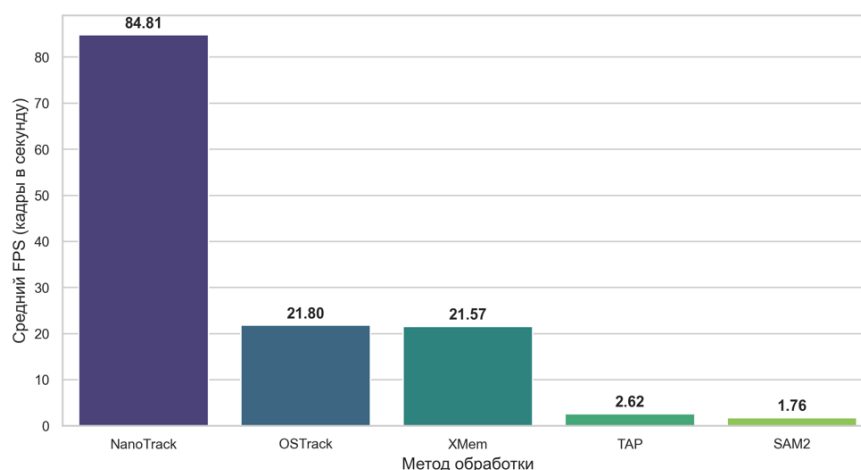


Рис. 3.3.1. FPS по методам

На рисунке 3.3.2 продемонстрирована зависимость времени обработки от разрешения исходного видео. Для всех методов кроме TAP время выполнения стабильно для разных расширений видео, однако растёт при переходе к формату 4K. Время работы метода TAP растёт стабильно в зависимости от роста качества (расширения) исходного видео.

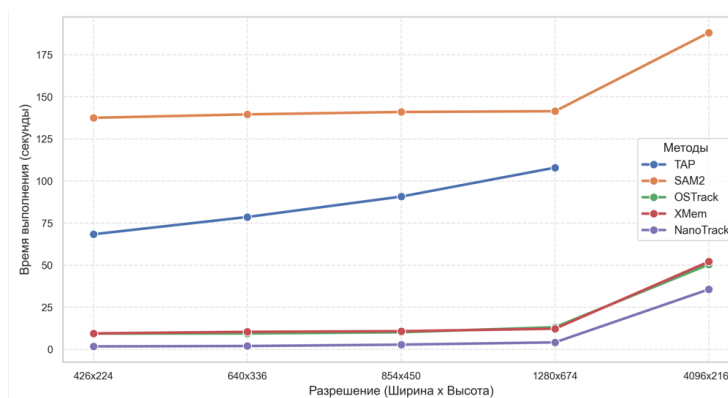


Рис.3.3.2. Зависимость времени обработки от расширения

Аналогичная ситуация с пиковым потреблением VRAM (рис.3.3.3, приложение 4): для всех методов оно стабильно, но у метода TAP растёт линейно расширению. Это может быть связано с тем, что внутри методов видео автоматически приводится к фиксированному низкому расширению для ускорения вычислений. Самыми легковесными оказались трекеры NanoTrack (около 915 МБ), алгоритм XMem и OTrack потребляет всего 1155 МБ видеопамяти. Модель SAM2 требует стабильных 2,9 ГБ VRAM, в то время как

TAP является самым ресурсоемким методом, потребляя от 4,8 до 6,2 ГБ, что служит причиной падения метода на больших видео.

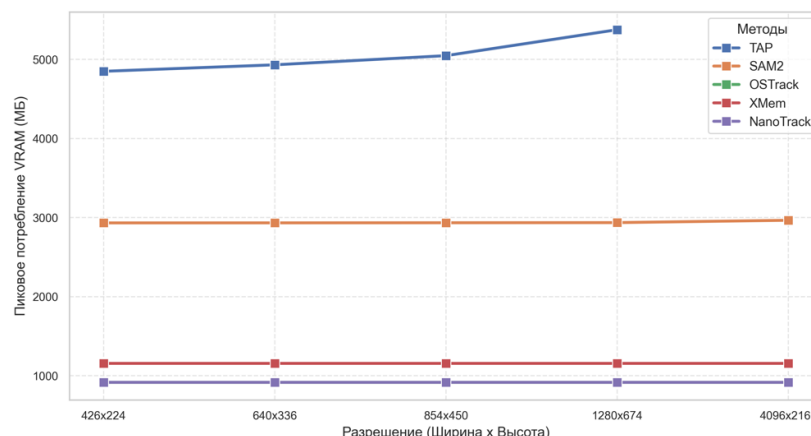


Рис. 3.3.3. Зависимость потребления VRAM от разрешения видео

Изменения оперативной памяти в процессе работы сильно колеблются и часто принимают отрицательные значения. Это объясняется нормальной работой встроенных механизмов Python, так как методы обработки видео работают на видеокарте и не используют RAM. На рисунке 3.3.4, построенном по таблице из приложения 3, можно увидеть насколько малы изменения RAM для каждого метода относительно VRAM, который уже описан выше.

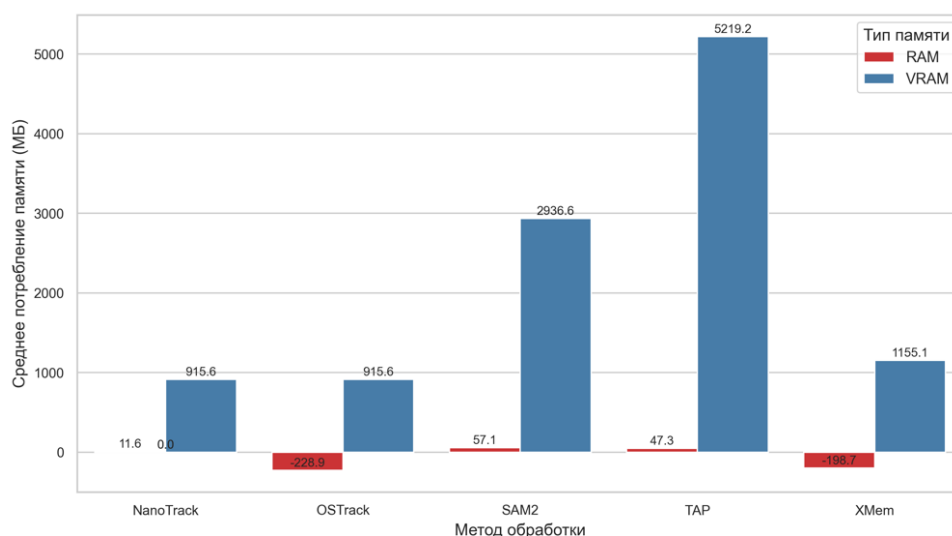


Рис. 3.3.4. Среднее потребление RAM и VRAM по методам

График построенный по таблице из приложения 5 показывает среднее качественных метрик по всем видео для каждого метода (рис. 3.3.5).

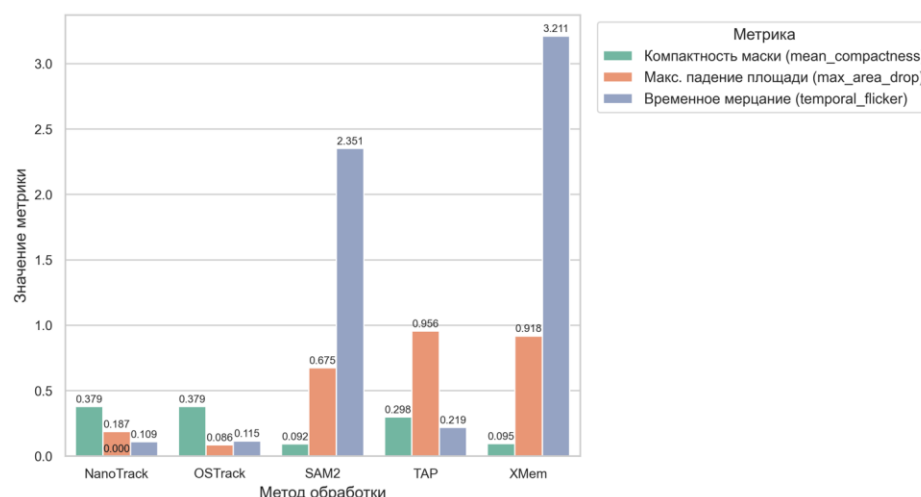


Рис. 3.3.5. Качественные метрики по методам

При сопоставлении формальных метрик качества алгоритмы класса Bounding Box (NanoTrack и OTrack) получают хорошие оценки стабильности площади маски и геометрической компактности, в то время как SAM2 и XMem показывают плохие результаты. Это обусловлено тем, что предсказания NanoTrack и OTrack это идеальные прямоугольники постоянной площади. Алгоритмы SAM2 и XMem строят подробные маски, изменение площади которых не только не является ошибкой, но и даже показывает точность трекинга контуров объекта. Для более корректного сравнения качества моделей проведем визуальную оценку результатов обработки.

3.4. Сравнение результатов обработки методов

Для сравнения будем использовать видео с серфингистом (объект). В течение видео он приближается к камере (меняется в размере), движется вверх по волне (двигается относительно кадра и меняет форму), а ближе к концу видео его накрывает волной (исчезает) (рис.3.4.1).



Рис. 3.4.1. Исходное видео

1. NanoTrack

От начала до конца видео объект попадает в bbox (рис. 3.4.2). Когда объект меняет свой размер, bbox также увеличивается, чтобы весь объект попадал в рамки. Когда объект исчезает с видео, bbox корректно исчезает.

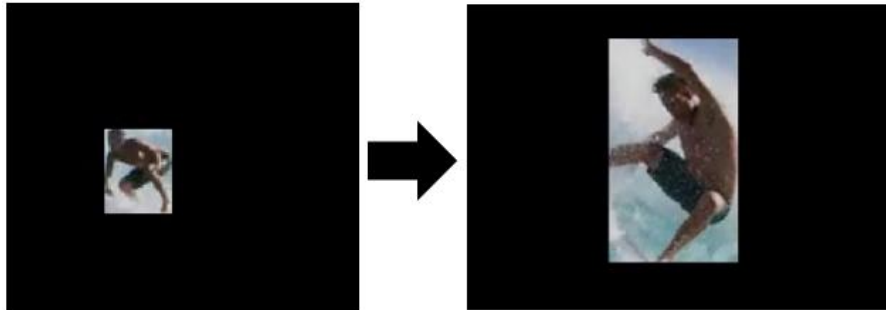


Рис. 3.4.2. Видео после обработки методом NanoTrack

2. OSTRack

На первом кадре строит правильный bbox, корректно следит за объектом. При увеличении объекта на видео рамка остается фиксированного размера и содержит только часть объекта (рис. 3.4.3). Под конец видео рамка теряет объект, но не исчезает, а продолжает метаться по кадру.

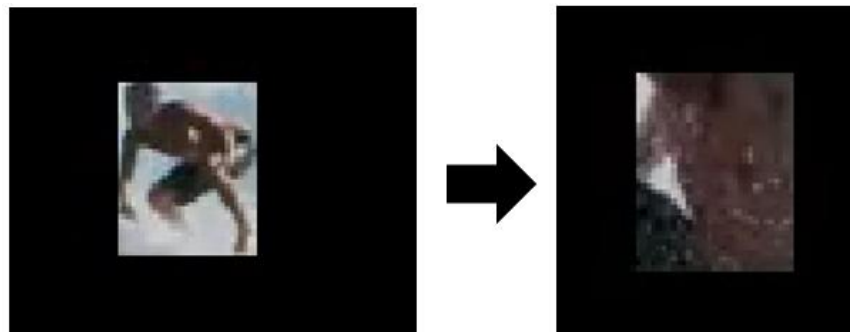


Рис. 3.4.3. Видео после обработки методом OSTRack

3. SAM2

Метод построил корректную маску хорошего качества с четкими границами. Хорошо отслеживается движение каждой частей фигуры (рис. 3.4.4). Корректно отображается исчезновение объекта.

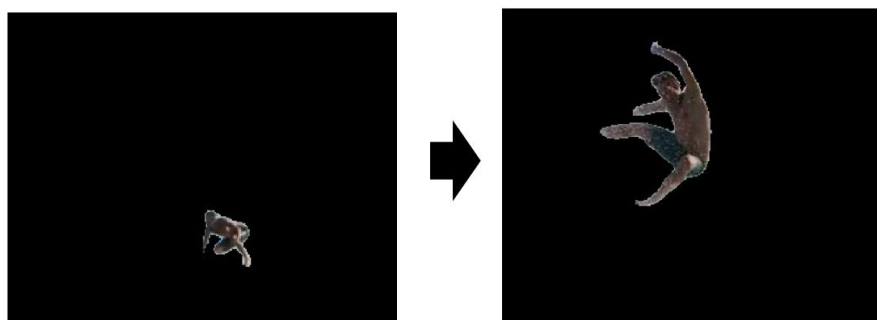


Рис. 3.4.4. Видео после обработки методом SAM2

4. TAP

На первых кадрах объект попадает в маску примерно на 80%. Однако уже на пятой секунде точки, за которыми следил TAP, изменились, и больше они не входят в маску (рис. 3.4.5). К шестой секунде треккинг совсем слетает. Объект быстро теряется.

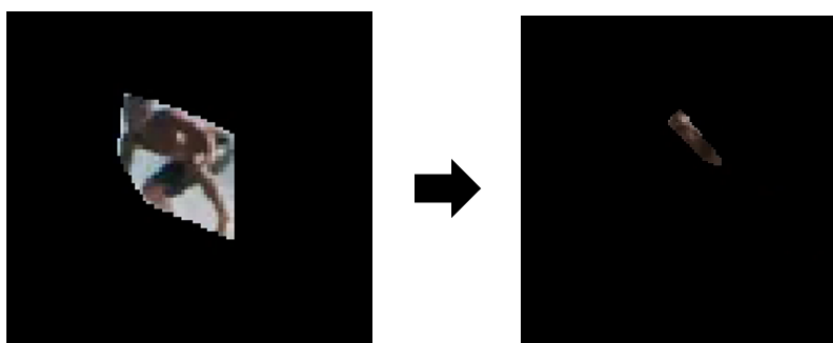


Рис. 3.4.5. Видео после обработки методом TAP

5. XMem

Маска метода XMem более точная чем маска TAP, но менее точная чем маска SAM2 (рис. 3.4.6). Метод хорошо отслеживает объект, не слетает, корректно отображает исчезновение.

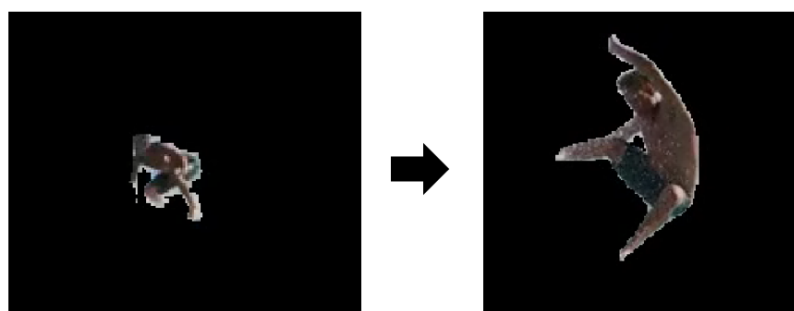


Рис. 3.4.6. Видео после обработки методом XMem

Таким образом, мы получили совсем другие показатели нежели могли ожидать по качественным метрикам. На таблице 3.4.1 представлено сравнение методов по качеству обработки, времени и потреблению ресурсов.

Таблица 3.4.1. Сравнение качества и применения методов

Метод	Тип	Поведение при изменении размера объекта	Поведение при исчезновении объекта	Качество выделения	Скорость	Ресурсы
NanoTrack	Bounding box	Отлично	Хорошо	Хорошо	Отлично	Отлично
OSTrack	Bounding box	Ужасно	Плохо	Нормально	Хорошо	Хорошо
SAM2	Маска	Отлично	Отлично	Отлично	Плохо	Плохо
TAP	Контур по точкам	Ужасно	Хорошо	Плохо	Плохо	Ужасно
XMem	Маска	Отлично	Хорошо	Хорошо	Хорошо	Хорошо

Несмотря на то что метод TAP очень плохо показал себя в контексте выделения маски объекта, его использование уместно при разработке сложных алгоритмов разделения видео на слои. Метод может следить за отдельными стабильными точками и подсказывать местоположение объекта для других моделей, которые в свою очередь будут строить маску.

ЗАКЛЮЧЕНИЕ

В ходе практики разработано веб-приложение для разделения видеопотока на слои, реализованное на базе Gradio [7]. В состав приложения интегрированы пять алгоритмов трекинга и сегментации на выбор: SAM2 Video, TAP, XMem, OTrack и NanoTrack. Для каждого метода реализован отдельный процессор, который получает от пользователя точки объекта на первом кадре, обрабатывает видео по чанкам и сохраняет результат в виде двух слоёв (объект и фон).

Проведён сравнительный эксперимент на 10 видеороликах с различными разрешениями, сценариями движения и количеством кадров. Зафиксированы временные характеристики, потребление оперативной и видеопамати, рассчитаны метрики качества без использования эталонных масок и проведен визуальный анализ результатов.

Точные маски были получены только с помощью методов SAM2 и XMem. SAM2 строит более точную и детальную маску, в то время как XMem быстрее и потребляет меньше ресурсов. NanoTrack обрабатывает видео кратно быстрее остальных методов и строит корректный bbox, в то время как OTrack работает дольше и возвращает видео хуже качеством. Метод TAP не применим к поставленной задаче, может использоваться только как прикладной трекинг стабильных точек, но не способен построить маску меняющегося объекта.

СПИСОК ИСТОЧНИКОВ

1. Segment-Anything / segment-anything-2 : репозиторий [Электронный ресурс] / владелец Segment-Anything. – Режим доступа: <https://github.com/Segment-Anything/segment-anything-2> (дата обращения: 20.05.2026). – Описание: код для вывода модели Meta Segment Anything Model 2 (SAM 2) для сегментации изображений и видео.
2. google-deepmind / tapnet : репозиторий [Электронный ресурс] / владелец google-deepmind. – Режим доступа: <https://github.com/google-deepmind/tapnet> (дата обращения: 20.05.2026). – Описание: реализация моделей отслеживания точек (Tracking Any Point), включая TAPIR, RoboTAP и другие.
3. hkchengrex / XMem : репозиторий [Электронный ресурс] / владелец hkchengrex. – Режим доступа: <https://github.com/hkchengrex/XMem> (дата обращения: 20.05.2026). – Описание: долгосрочная сегментация видеообъектов с моделью памяти Аткинсона–Шиффрина (ECCV 2022).
4. botaoye / OTrack : репозиторий [Электронный ресурс] / владелец botaoye. – Режим доступа: <https://github.com/botaoye/OTrack> (дата обращения: 20.05.2026). – Описание: официальная реализация OTrack — однопоточной системы для совместного обучения признаков и моделирования отношений при отслеживании (ECCV 2022).
5. HonglinChu / NanoTrack : репозиторий [Электронный ресурс] / владелец HonglinChu. – Режим доступа: <https://github.com/HonglinChu/NanoTrack> (дата обращения: 20.05.2026). – Описание: лёгкая и высокоскоростная сеть для отслеживания объектов на мобильных устройствах (NCNN, PyTorch).

6. visionml / pytracking : репозиторий [Электронный ресурс] / владелец visionml. – Режим доступа: <https://github.com/visionml/pytracking> (дата обращения: 20.05.2026). – Описание: библиотека для визуального отслеживания объектов и сегментации видео на PyTorch, включая трекеры ATOM, DiMP, PrDiMP, ToMP, TaMOs и другие.
7. LiSSk0 / AI-video-processor : репозиторий [Электронный ресурс] / владелец LiSSk0. – Режим доступа: <https://github.com/LiSSk0/AI-video-processor> (дата обращения: 20.05.2026). – Описание: интерфейс для разделения слоёв видео и построения карт глубины с использованием ИИ.

Приложения

Таблица 1. Время выполнения обработки видео (сек)

Название видеоролика	OTrack	NanoTrack	SAM2	TAP	XMem
457918_Birds_Flying_1920x1080.mp4	21,36	11,47	176,81	271,08	21,44
Birds_02_4K_res.mp4	50,38	35,63	188,15		52,09
Birds_02_4K_res_1280x720.mp4	13,15	4,15	141,46	107,93	12,25
Birds_02_4K_res_426x240.mp4	9,36	1,74	137,53	68,36	9,47
Birds_02_4K_res_640x360.mp4	9,43	1,97	139,58	78,61	10,40
Birds_02_4K_res_854x480.mp4	10,15	2,79	141,00	90,74	10,79
field.MOV	13,01	2,385	188,18	138,46	11,12
istockphoto-1140675444-640_adpp_is.mp4	9,39	2,335	133,34	84,33	9,72
istockphoto-1304919722-640_adpp_is.mp4	13,50	3,65	184,98	150,89	13,46
ivan.MOV	11,59	6,41	142,53		13,37

Таблица 2. Средняя частота обработки кадров (FPS)

Название видеоролика	NanoTrack	OTrack	SAM2	TAP	Xmem
457918_Birds_Flying_1920x1080.mp4	24,39	13,10	1,58	1,03	13,05
Birds_02_4K_res.mp4	7,24	5,12	1,37		4,95
Birds_02_4K_res_1280x720.mp4	62,14	19,61	1,82	2,39	21,04
Birds_02_4K_res_426x240.mp4	147,96	27,54	1,87	3,77	27,24
Birds_02_4K_res_640x360.mp4	130,57	27,34	1,84	3,28	24,80
Birds_02_4K_res_854x480.mp4	92,29	25,40	1,82	2,84	23,88
field.MOV	146,04	26,74	1,84	2,51	31,26
istockphoto-1140675444-640_adpp_is.mp4	104,79	26,07	1,83	2,90	25,18
istockphoto-1304919722-640_adpp_is.mp4	93,02	25,17	1,83	2,25	25,25
ivan.MOV	39,57	21,89	1,78		18,99

Таблица 3. RAM Delta, МБ

Название видеоролика	OTrack (RAM Delta, МБ)	NanoTrack (RAM Delta, МБ)	SAM2 (RAM Delta, МБ)	TAP (RAM Delta, МБ)	XMem (RAM Delta, МБ)
457918_Birds_Flying_1920x1080.mp4	- 951,6230 469	2,5839843 75	102,6718 75	481,1425 781	45,60546 875
Birds_02_4K_res.mp4	29,36523 438	- 78,789062 5	7,683593 75		- 273,4335 938
Birds_02_4K_res_1280x720.mp4	- 185,0839 844	47,025390 63	29,51953 125	79,12890 625	- 229,8769 531

Birds_02___4K_res_426x240. mp4	19,17382 813	1,5917968 75	- 48,19140 625	- 155,5683 594	- 254,9023 438
Birds_02___4K_res_640x360. mp4	- 189,0527 344	32,875	- 65,78906 25	11,10546 875	- 258,4492 188
Birds_02___4K_res_854x480. mp4	- 123,0234 375	35,074218 75	-6,90625	- 83,28710 938	- 255,7734 375
field.MOV	- 88,44726 563	26,685546 88	530,0664 063	- 5,242187 5	11,26562 5
istockphoto-1140675444- 640_adpp_is.mp4	8,412109 375	16,628906 25	- 368,1757 813	- 132,3320 313	- 259,8886 719
istockphoto-1304919722- 640_adpp_is.mp4	- 248,2265 625	32,900390 63	458,8125	183,4414 063	- 253,2363 281
ivan.MOV	- 560,9296 875	- 0,3554687 5	- 68,37890 625		- 258,2812 5

Таблица 4. VRAM Пик, МБ

Название видеоролика	NanoTrack (VRAM Пик, МБ)	OTrack (VRAM Пик, МБ)	SAM2 (VRAM Пик, МБ)	TAP (VRAM Пик, МБ)	XMem (VRAM Пик, МБ)
457918_Birds_Flying_1920x1080.mp4	915,5751953	915,5751953	2924,45459	6218,287842	1155,986816
Birds_02___4K_res.mp4	915,5751953	915,5751953	2966,29834		1155,049316
Birds_02___4K_res_1280x720.mp4	915,5751953	915,5751953	2936,544434	5377,113281	1155,049316
Birds_02___4K_res_426x240.mp4	915,5751953	915,5751953	2932,912598	4850,717773	1155,049316
Birds_02___4K_res_640x360.mp4	915,5751953	915,5751953	2933,368652	4932,720215	1155,049316
Birds_02___4K_res_854x480.mp4	915,5751953	915,5751953	2934,953125	5048,722656	1155,049316
field.MOV	915,5751953	915,5751953	2933,106934	4885,269043	1155,049316
istockphoto-1140675444-	915,5751953	915,5751953	2933,813965	5012,72168	1155,049316

640_adpp_is.mp4					
istockphoto-1304919722-640_adpp_is.mp4	915,5751953	915,5751953	2933,813965	5428,349121	1155,049316
ivan.MOV	915,5751953	915,5751953	2936,544434		1155,049316

Таблица 5. Средние значения качественных параметров

Метод	Компактность маски	Макс. падение площади	Временное мерцание
NanoTrack	0,378732738	0,187475815	0,109074218
OSTrack	0,378587228	0,085689593	0,114963995
SAM2	0,092030145	0,674880351	2,351133382
TAP	0,29821218	0,956022099	0,219098198
XMem	0,095317791	0,917855192	3,21120029